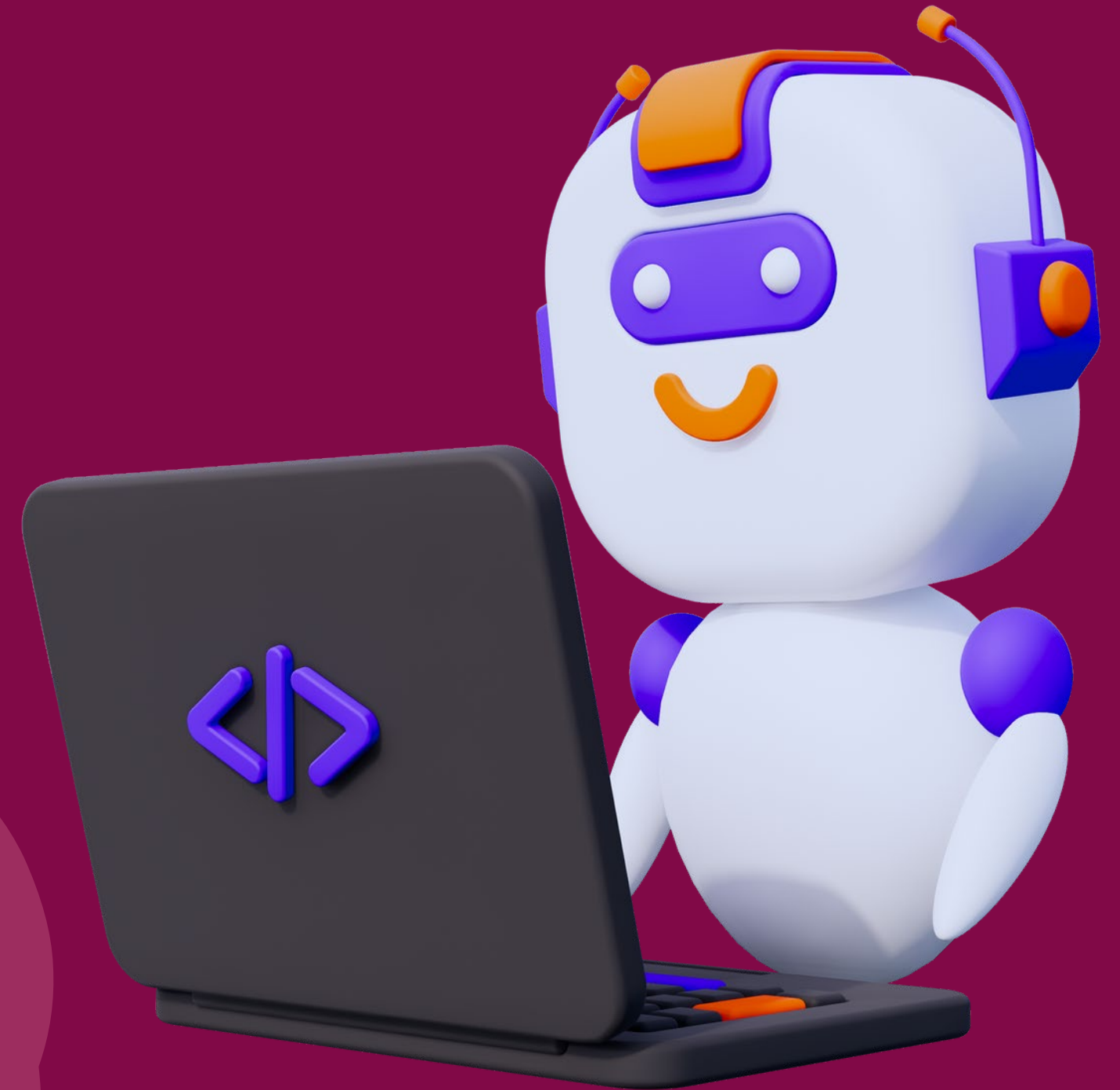
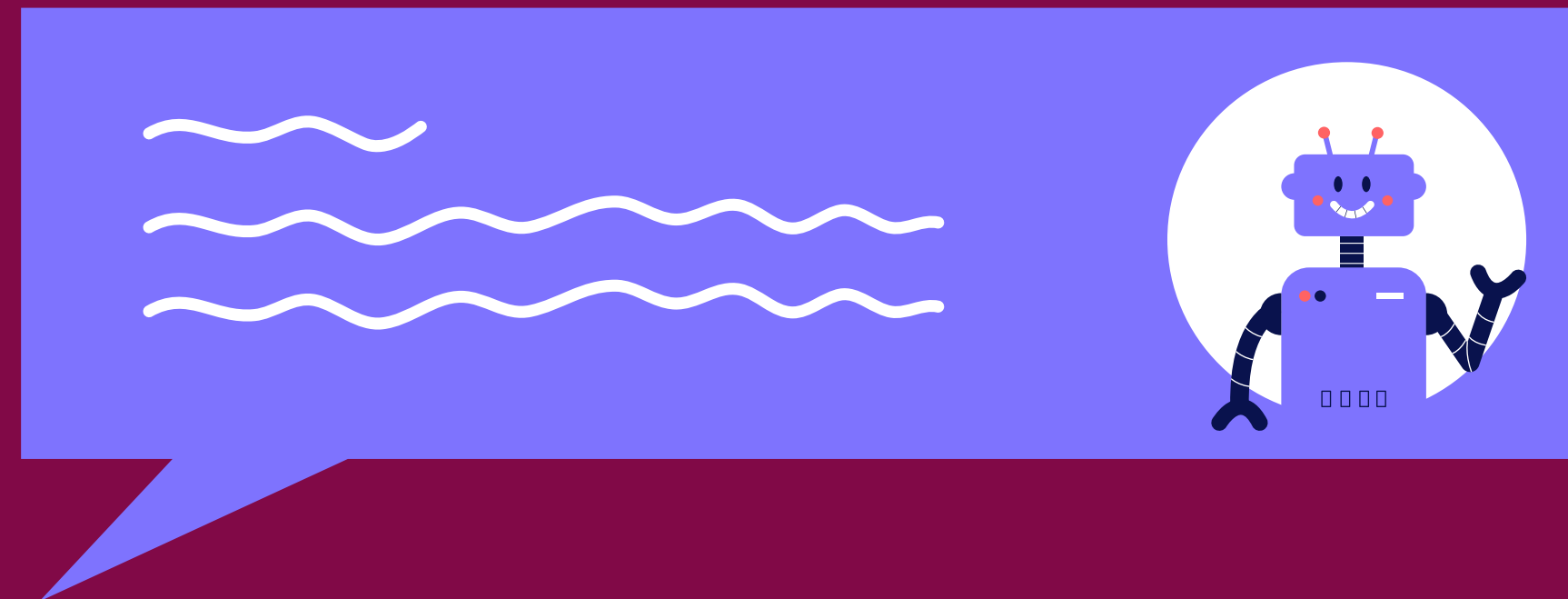
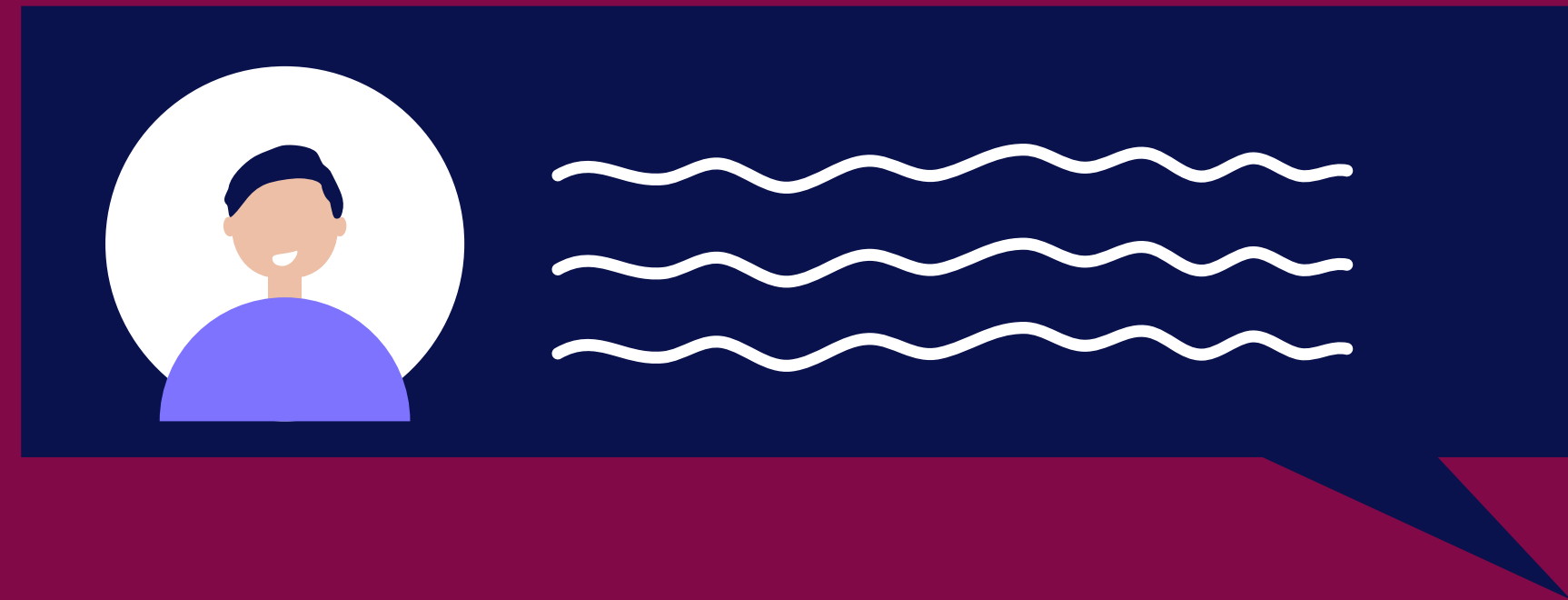


AI Tools in Software Development

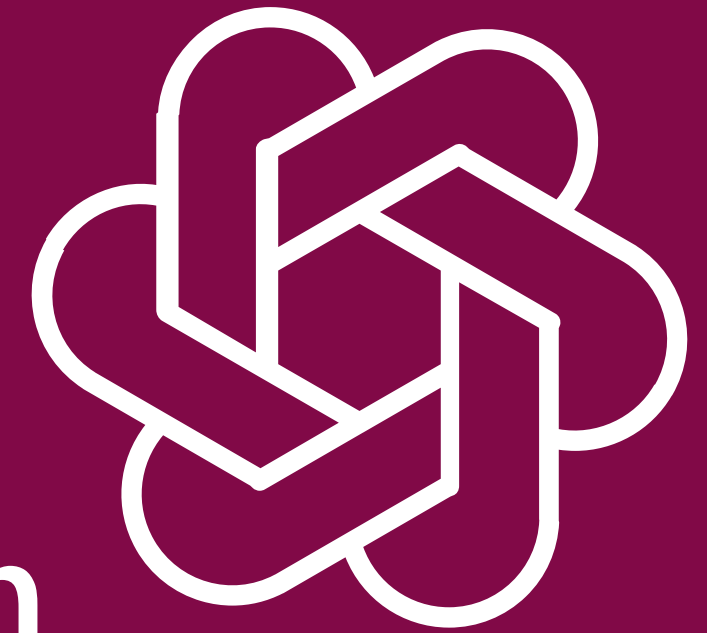






ChatGPT

Introduction

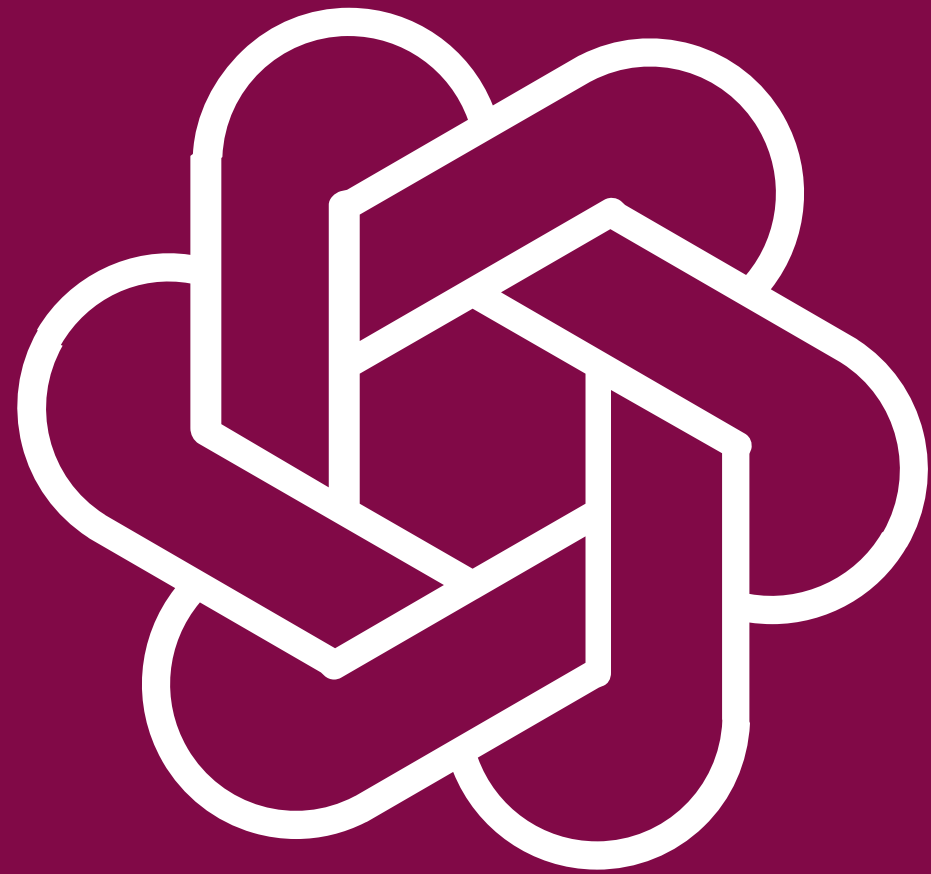


Overview of ChatGPT by OpenAI

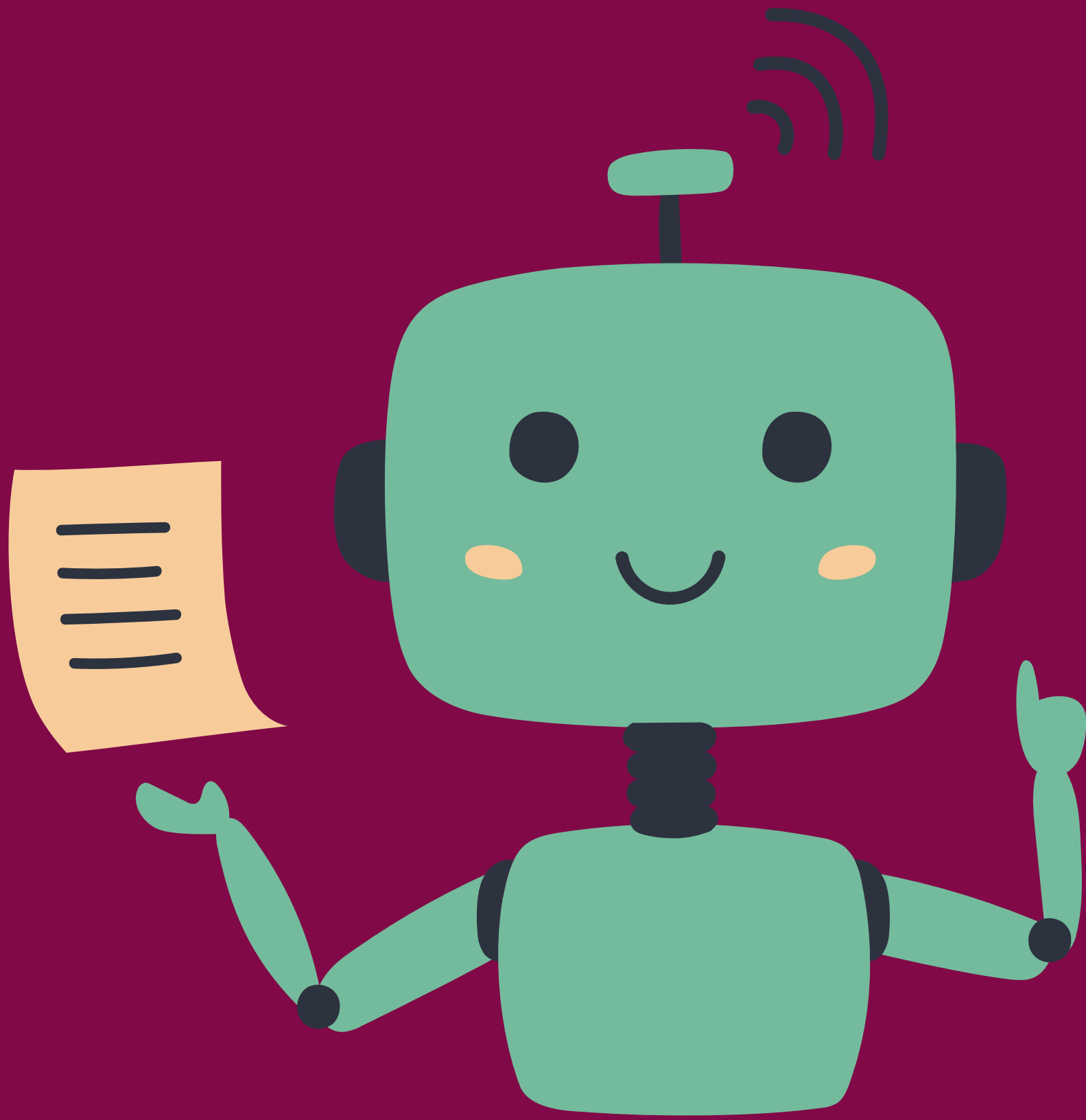
Capabilities and limitations of ChatGPT

Example use cases in software development

Uses for ChatGPT



- Generating code snippets and templates
- Assisting in debugging and problem-solving
- Enhancing documentation and comments
- Brainstorming and ideation for new features



How to Use ChatGPT to Generate Ideas

Examples of prompts for idea generation

Integrating ChatGPT into your development workflow

Improving User Retention for a Mobile Game

Developer: "We need to improve user retention for our mobile game."

ChatGPT: "What are some current features that users enjoy?"

Developer: "Users enjoy the daily challenges and rewards."

ChatGPT: "How about adding weekly tournaments or special events?"

Improving User Retention for a Mobile Game

Developer: "How can we implement weekly tournaments?"

ChatGPT: "Create a leaderboard system where players compete weekly, offering unique rewards for top performers."

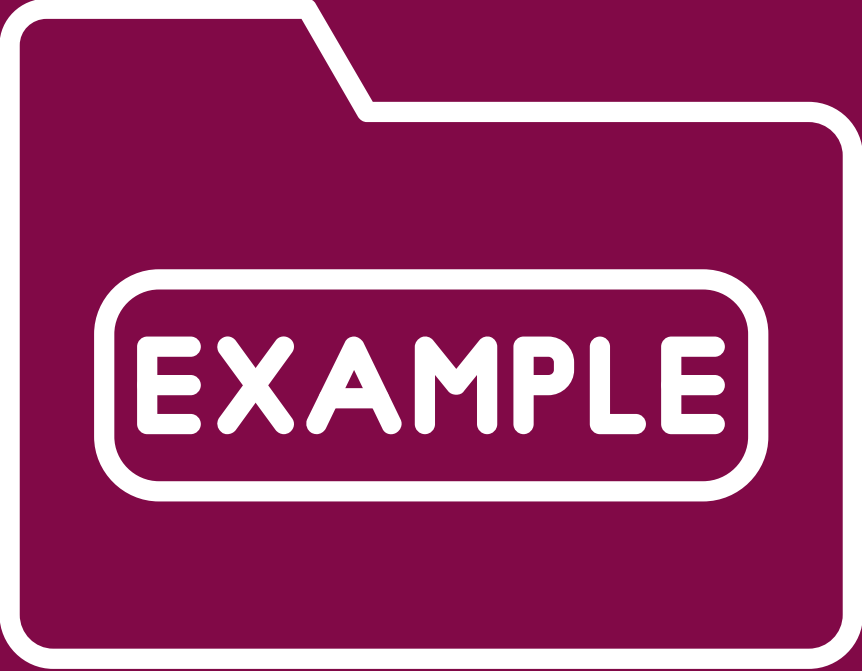
Developer: "What challenges might we face?"

ChatGPT: "Ensuring fair competition. Use skill-based matchmaking and anti-cheat mechanisms."

Improving User Retention for a Mobile Game

Developer: "What are the key steps for integration?"

ChatGPT: "1. Design tournament rules and rewards. 2. Develop leaderboard and matchmaking. 3. Test for balance. 4. Launch beta for feedback."



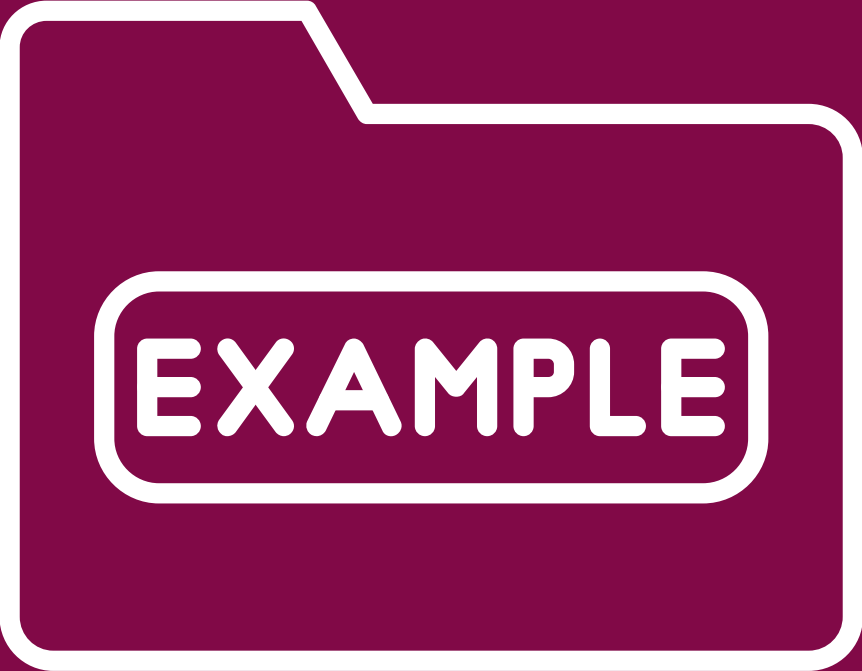
EXAMPLE

Task: Write a Python function to calculate the average of a list of numbers.

AI-Generated Code with Prompt:

"Write a function to find the average of numbers."

```
def calculate_average(numbers):  
    return sum(numbers) / len(numbers)  
  
# Test case  
print(calculate_average([1, 2, 3, 4, 5]))
```



EXAMPLE

Task: Write a Python function to calculate the average of a list of numbers.

AI-Generated Code with Prompt:

"Write a function to find the average of numbers."

```
def calculate_average(numbers):  
    return sum(numbers) / len(numbers)  
  
# Test case  
print(calculate_average([1, 2, 3, 4, 5]))
```

The AI correctly writes a function to calculate the average, but what if the list is empty? The current function would raise a `'ZeroDivisionError'`.

EXAMPLE

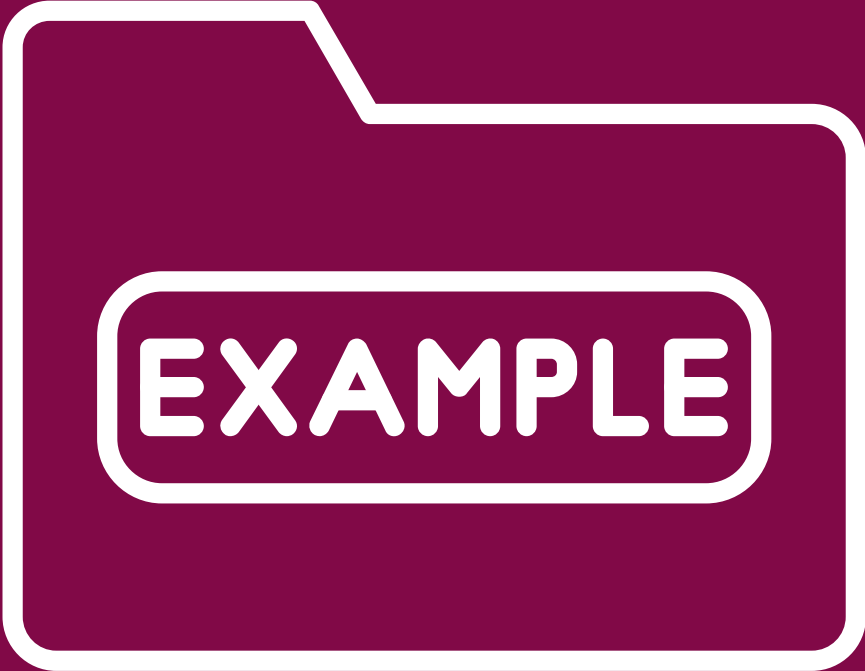
Write a Python function that takes a list of numbers and returns the average of the numbers.

numbers = [1, 2, 3, 4, 5]
Understand the prompt:

"Write a Python function that takes a list of numbers and returns the average of the numbers."

```
def calculate_average(numbers):  
    return sum(numbers) / len(numbers)  
  
# Test case  
print(calculate_average([1, 2, 3, 4, 5]))
```

AI correctly writes a function to calculate the average, but what if the list is empty? The current function would raise a `ZeroDivisionError`.



EXAMPLE

Task: Write a Python function to calculate the average of a list of numbers.

AI-Generated Code with CLEAR Prompt:

"Write a Python function to calculate the average of a list of numbers. Ensure it handles an empty list by returning 'None' or an appropriate message."

```
def calculate_average(numbers):  
    if not numbers:  
        return None # or "List is empty"  
    return sum(numbers) / len(numbers)  
  
# Test cases  
print(calculate_average([1, 2, 3, 4, 5])) # Should print 3.0  
print(calculate_average([])) # Should print None or "List is empty"
```

EXAMPLE

Task: Write a Python function to calculate the average of a list of numbers.

AI-Generated Code and Prompt:

"Write a Python function to calculate the average of a list of numbers. The function should handle an empty list by returning 'None' and an appropriate message."

```
def calculate_average(numbers):
    if not numbers:
        return None # or "List is empty"
    return sum(numbers) / len(numbers)

# Test cases
print(calculate_average([1, 2, 3])) # Should print 3.0
print(calculate_average([])) # Should print "List is empty"
```




Task: Write a Python function to calculate the average of a list of numbers.

AI-Generated Code with CLEAR Prompt:

"Write a Python function to calculate the average of a list of numbers. Ensure it handles an empty list by returning `None` or an appropriate message."

```
def calculate_average(numbers):  
    if not numbers:  
        return None # or "List is empty"  
    return sum(numbers) / len(numbers)  
  
# Test cases  
print(calculate_average([1, 2, 3, 4, 5])) # Should print 3.0  
print(calculate_average([])) # Should print None or "List is empty"
```

Vague instructions can lead to incomplete or incorrect solutions. By providing clear and precise directions, the AI can generate more accurate and robust code.

Integrating ChatGPT into your development workflow

